Lab 4: Spring Security

Secure the Todo application with authentication, role-based authorization and CSRF protection

Open and continue project from Lab 3.

What you will learn

- Add spring-boot-starter-security and thymeleaf-extras-springsecurity6 to the pom.xml file.
- Implement custom authentication and authorization with MySQL using User entity, UserRepository and CustomUserDetailsService.
- Configure SecurityFilterChain in Spring Boot 3 lambda DSL style with PasswordEncoder and initial account seeding.
- Enforce method-level security on RESTful controllers using @EnableMethodSecurity and @PreAuthorize.
- Enforce role-based access control: regular users can view/create/edit; only ADMIN can delete todos.
- Enable HTTP Basic authentication to secure RESTful API endpoints for Postman.
- Configure CSRF protection: keep enabled for Thymeleaf forms while exempting /api/v1/** for API testing.
- Use Thymeleaf Security tags (sec:authentication, sec:authorize) to conditionally render UI elements.

Architecture Benefit: Security at the Filter Boundary

Spring Security intercepts requests before they reach your controllers.

Your existing Todo entity, TodoRepository and TodoService remain completely unchanged while gaining authentication, role-based access control and CSRF defense.

✓ Step 1. Add Spring Security to the pom.xml file

Open the pom.xml file and add these dependencies

- spring-boot-starter-security
- thymeleaf-extras-springsecurity6

pom.xml: add dependencies inside <dependencies> block

```
<!-- Spring Security Starter -->
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-security</artifactId>
</dependency>

<!-- Thymeleaf Security 6 dialect for UI authorization tags -->
<dependency>
    <groupId>org.thymeleaf.extras</groupId>
    <artifactId>thymeleaf-extras-springsecurity6</artifactId>
</dependency>
```

- spring-boot-starter-security:
  - adds authentication filter chain
  - password hashing
  - authorization checks.
  - Default login form provided automatically.
- thymeleaf-extras-springsecurity6:
  - provides sec:authorize and sec:authentication attributes in HTML templates.
- Reload Maven dependencies in your IDE after updating pom.xml.

✓ Step 2. Project structure overview

Review your updated project layout.

We will add the User entity, UserRepository, CustomUserDetailsService and config/SecurityConfig.java files.

```
src/main/java/com/example/todolist/
├── config/
│   └── SecurityConfig.java          # SecurityFilterChain, PasswordEncoder & user seed
├── controller/
│   ├── TodoController.java
│   └── api/v1/
│       └── TodoRestController.java # REST API with @PreAuthorize
├── dto/
│   └── TodoRequest.java
├── exception/
│   └── TodoNotFoundException.java
├── model/
│   ├── Todo.java
│   └── User.java                    # user credentials and role entity
├── repository/
│   ├── TodoRepository.java
│   └── UserRepository.java          # findByUsername for authentication
├── service/
│   ├── CustomUserDetailsService.java # loads user details from MySQL
│   └── TodoService.java
└── TodolistApplication.java
```

✓ Step 3. Configure User Entity, Repository and CustomUserDetailsService for MySQL

Create these files:

1. The User entity mapped to MySQL table 'users'
2. The UserRepository interface to interact with MySQL table 'users'
3. The CustomUserDetailsService implementing Spring Security's UserDetailsService.

src/main/java/com/example/todolist/model/User.java

```
package com.example.todolist.model;

import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.Table;

@Entity
@Table(name = "users")
public class User {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, unique = true)
    private String username;

    @Column(nullable = false)
    private String password;

    @Column(nullable = false)
    private String role; // "USER" or "ADMIN"

    public User() {
    }

    public User(String username, String password, String role) {
        this.username = username;
        this.password = password;
        this.role = role;
    }

    public Long getId() {
        return id;
    }

    public void setId(Long id) {
        this.id = id;
    }

    public String getUsername() {
        return username;
    }
```

```
    public void setUsername(String username) {
        this.username = username;
    }

    public String getPassword() {
        return password;
    }

    public void setPassword(String password) {
        this.password = password;
    }

    public String getRole() {
        return role;
    }

    public void setRole(String role) {
        this.role = role;
    }
}
```

src/main/java/com/example/todolist/repository/UserRepository.java

```
package com.example.todolist.repository;

import com.example.todolist.model.User;
import org.springframework.data.jpa.repository.JpaRepository;

import java.util.Optional;

public interface UserRepository extends JpaRepository<User, Long> {
    Optional<User> findByUsername(String username);
}
```

We use the Optional<T> wrapper type that forces the caller to handle the case where a value is missing, reducing the risk of NullPointerException.

src/main/java/com/example/todolist/service/CustomUserDetailsService.java

```
package com.example.todolist.service;

import com.example.todolist.model.User;
import com.example.todolist.repository.UserRepository;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.security.core.userdetails.UserDetailsService;
import org.springframework.security.core.userdetails.UsernameNotFoundException;
import org.springframework.stereotype.Service;

@Service
public class CustomUserDetailsService implements UserDetailsService {

    private final UserRepository userRepository;

    public CustomUserDetailsService(UserRepository userRepository) {
        this.userRepository = userRepository;
    }

    @Override
    public UserDetails loadUserByUsername(String username) throws UsernameNotFoundException
{
        User user = userRepository.findByUsername(username)
                .orElseThrow(() -> new UsernameNotFoundException("User not found: " +
username));

        return org.springframework.security.core.userdetails.User.builder()
                .username(user.getUsername())
                .password(user.getPassword())
                .roles(user.getRole())
                .build();
    }
}
```

- UserDetailsService: core Spring Security interface for loading user data by username during authentication.
- loadUserByUsername: queries MySQL via userRepository.findByUsername(username). If found, returns UserDetails with BCrypt-hashed password and roles.
- roles(user.getRole()): Spring automatically prepends ROLE_ (e.g. USER -> ROLE_USER).

✓ Step 4. Configure SecurityConfig, method security and seed initial users

Create package config and add new SecurityConfig.java file.

SecurityConfig.java file is a central point for processing information related to security.

In this file we can configure:

- PasswordEncoder to hash password
- Enable method-level security with @EnableMethodSecurity
- Define the SecurityFilterChain
- Automatically seed initial users into MySQL.

src/main/java/com/example/todolist/config/SecurityConfig.java

```
package com.example.todolist.config;

import com.example.todolist.model.User;
import com.example.todolist.repository.UserRepository;
import org.springframework.boot.CommandLineRunner;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.http.HttpMethod;
import org.springframework.security.config.Customizer;
import
org.springframework.security.config.annotation.method.configuration.EnableMethodSecurity;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.config.annotation.web.configuration.EnableWebSecurity;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.web.SecurityFilterChain;

@Configuration
@EnableWebSecurity
@EnableMethodSecurity
public class SecurityConfig {

    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }

    @Bean
    public CommandLineRunner initUsers(UserRepository userRepository, PasswordEncoder
encoder) {
        return args -> {
            if (userRepository.count() == 0) {
                userRepository.save(new User("user", encoder.encode("password123"),
"USER"));
                userRepository.save(new User("admin", encoder.encode("admin123"), "ADMIN"));
            }
        };
    }
```

```
    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
        http
            .csrf(csrf -> csrf
                .ignoringRequestMatchers("/api/v1/**")
            )
            .authorizeHttpRequests(auth -> auth
                .requestMatchers(HttpMethod.POST, "/todos/*/delete").hasRole("ADMIN")
                .requestMatchers(HttpMethod.GET, "/api/v1/todos/**").hasAnyRole("USER",
"ADMIN")
                .requestMatchers(HttpMethod.POST, "/api/v1/todos/**").hasAnyRole("USER",
"ADMIN")
                .requestMatchers(HttpMethod.PUT, "/api/v1/todos/**").hasAnyRole("USER",
"ADMIN")
                .requestMatchers(HttpMethod.DELETE, "/api/v1/todos/**").hasRole("ADMIN")
                .requestMatchers("/todos/**").authenticated()
                .requestMatchers("/api/v1/**").authenticated()
                .anyRequest().permitAll()
            )
            .formLogin(form -> form
                .defaultSuccessUrl("/todos", true)
                .permitAll()
            )
            .httpBasic(Customizer.withDefaults()) // HTTP Basic Auth username:password
(Base64)
            .logout(logout -> logout
                .logoutSuccessUrl("/login")
                .permitAll()
            );

        return http.build();
    }
}
```

- @EnableMethodSecurity: activates method-level security annotations like @PreAuthorize on controllers and services.
- initUsers: CommandLineRunner that automatically inserts default user and admin accounts into MySQL with BCrypt-hashed passwords on startup if the table is empty.
- CustomUserDetailsService: Spring Security automatically discovers your @Service and uses it for both Form Login and HTTP Basic authentication.
- .ignoringRequestMatchers("/api/v1/**"): disables CSRF for REST endpoints so tools like Postman can POST, PUT, DELETE freely.
- hasRole("ADMIN"): restricts delete actions strictly to users with the ADMIN role.
- httpBasic(): enables HTTP Basic authentication for REST API clients.
- CSRF (Cross-Site Request Forgery) is a type of attack in which an attacker tricks a user's browser into sending a "forged" request to a server (while the user is logged in).

✓ Step 5. Secure RESTful API controller with method security

Update src/main/java/com/example/todolist/controller/api/v1/TodoRestController.java to apply @PreAuthorize annotations.

This creates defense-in-depth: authorization is checked both at the URL filter chain and right before the controller method executes.

src/main/java/com/example/todolist/controller/api/v1/TodoRestController.java

```
package com.example.todolist.controller.api.v1;

import com.example.todolist.dto.TodoRequest;
import com.example.todolist.model.Todo;
import com.example.todolist.service.TodoService;
import jakarta.validation.Valid;
import org.springframework.security.access.prepost.PreAuthorize;
import org.springframework.web.bind.annotation.DeleteMapping;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.PutMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

import java.util.List;

@RestController
@RequestMapping("/api/v1/todos")
public class TodoRestController {

    private final TodoService todoService;

    public TodoRestController(TodoService todoService) {
        this.todoService = todoService;
    }

    @GetMapping
    public List<Todo> list() {
        return todoService.findAll();
    }

    @GetMapping("/{id}")
    public Todo getOne(@PathVariable Long id) {
        return todoService.findById(id);
    }

    @PostMapping
    @PreAuthorize("hasAnyRole('USER', 'ADMIN')")
    public Todo create(@Valid @RequestBody TodoRequest request) {
        return todoService.create(request);
    }

    @PutMapping("/{id}")
    @PreAuthorize("hasAnyRole('USER', 'ADMIN')")
    public Todo update(@PathVariable Long id,
                       @Valid @RequestBody TodoRequest request) {
        return todoService.update(id, request);
```

```
    }

    @DeleteMapping("/{id}")
    @PreAuthorize("hasRole('ADMIN')")
    public void delete(@PathVariable Long id) {
        todoService.deleteById(id);
    }
}
```

- @PreAuthorize("hasRole('ADMIN')"): restricts the delete endpoint strictly to users with the ADMIN role at the method invocation level.
- @PreAuthorize("hasAnyRole('USER', 'ADMIN')"): ensures only authenticated users with USER or ADMIN role can create or modify todos.
- Defense-in-depth: even if a URL matcher is misconfigured, method security intercepts unauthorized execution and throws AccessDeniedException (HTTP 403).
- Other types
  - @PreAuthorize("#userId == authentication.principal.id or hasRole('ADMIN')"): users can only delete their own to-do items, whereas admins can delete everything. SecurityConfig cannot handle this because it only recognizes URLs and lacks knowledge of business logic.
  - @PreAuthorize("@todoSecurity.isOwner(#id, authentication.name)"): perform a check based on a specific object. Call the `todoSecurity` bean to verify ownership.
- Safety when refactoring URLs: if you change a URL from "/todos/{id}/delete" to "/api/v2/todos/{id}", the rule in SecurityConfig will no longer match, creating a hidden security vulnerability. In contrast, @PreAuthorize is tied to the method itself, so it remains valid.

✓ Step 6. Update Thymeleaf template with Security Dialect

Update the src/main/resources/templates/todos/list.html file.

Display the logged-in user banner, logout button and hide the Delete button from non-admin users.

src/main/resources/templates/todos/list.html

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org"
      xmlns:sec="http://www.thymeleaf.org/extras/spring-security">
<head>
    <meta charset="UTF-8"/>
    <title>Todos</title>
    <style>
        body { font-family: Segoe UI, sans-serif; margin: 24px; }
        .user-bar { background: #f0f4f8; padding: 10px 16px; margin-bottom: 16px; border-
radius: 4px; display: flex; justify-content: space-between; align-items: center; }
        table { border-collapse: collapse; width: 100%; }
        th, td { border: 1px solid #ccc; padding: 8px; text-align: left; }
        a, button { margin-right: 8px; }
    </style>
</head>
<body>
<div class="user-bar">
    <div>
        Logged in as: <strong sec:authentication="name">User</strong>
        (<span sec:authentication="principal.authorities">[ROLE_USER]</span>)
    </div>
    <form th:action="@{/logout}" method="post" style="display:inline">
        <button type="submit">Logout</button>
    </form>
</div>
<h1>Todo list</h1>
<p><a th:href="@{/todos/new}">New todo</a></p>
<table>
    <thead>
    <tr>
        <th>ID</th><th>Title</th><th>Due</th><th>Done</th><th></th>
    </tr>
    </thead>
    <tbody>
    <tr th:each="t : ${todos}">
        <td th:text="${t.id}">1</td>
        <td th:text="${t.title}">title</td>
        <td th:text="${t.dueDate}">2026-01-01</td>
        <td th:text="${t.completed} ? 'Yes' : 'No'">No</td>
        <td>
            <a th:href="@{/todos/{id}/edit(id=${t.id})}">Edit</a>
            <!-- Delete button is ONLY rendered for users with ADMIN role -->
            <form sec:authorize="hasRole('ADMIN')"
                  th:action="@{/todos/{id}/delete(id=${t.id})}"
                  method="post" style="display:inline">
                <button type="submit">Delete</button>
            </form>
        </td>
    </tr>
    </tbody>
</table>
</body>
</html>
```

- xmlns:sec: registers the Spring Security XML namespace for Thymeleaf.
- sec:authentication="name": prints the currently authenticated username.
- sec:authorize="hasRole('ADMIN')": completely hides the Delete button and form from users who lack the ADMIN role.
- th:action="@{/logout}": generates the logout URL and automatically injects the required hidden CSRF input.

✓ Step 7. Test form login and role authorization in browser

Start the application and test login, role-based visibility and logout in your browser.

1. Run TodolistApplication.java and verify MySQL table: open MySQL Workbench and run: USE todolist_db; SELECT * FROM users; -> verify that user and admin records exist with BCrypt hashed passwords.
2. Navigate to http://localhost:8080/todos in your browser. Notice you are automatically redirected to the default Spring Security login page at http://localhost:8080/login
3. Log in with user credentials: username 'user', password 'password123'. You are redirected to /todos.
4. Notice the user bar shows 'Logged in as: user ([ROLE_USER])'. Look at the table: the Delete button is completely hidden!
5. Click 'Logout' in the top bar. You are redirected to /login?logout
6. Now log in with admin credentials: username 'admin', password 'admin123'.
7. Notice the user bar shows 'Logged in as: admin ([ROLE_ADMIN, ROLE_USER])' and the Delete button is now visible.
8. Click 'Delete' on a todo: the item is successfully deleted.

✓ Step 8. Test REST API with HTTP Basic Authentication in Postman

Verify that API endpoints require credentials and enforce role restrictions when called via HTTP Basic.

1. Open Postman and send GET http://localhost:8080/api/v1/todos with no authorization: server responds with HTTP 401 Unauthorized.
2. In Postman, switch to the 'Authorization' tab, select Type: 'Basic Auth', and enter username: 'user', password: 'password123'.
3. Send GET http://localhost:8080/api/v1/todos: server responds with HTTP 200 OK and returns the JSON list.
4. Keep 'user' Basic Auth credentials and send DELETE http://localhost:8080/api/v1/todos/1: server responds with HTTP 403 Forbidden (access denied by both URL rule and @PreAuthorize!).
5. Change credentials in the Authorization tab to username: 'admin', password: 'admin123'.
6. Resend DELETE http://localhost:8080/api/v1/todos/1: server responds with HTTP 200 OK and the todo is successfully removed from the database.

✓ Step 9. Summary

- pom.xml contains spring-boot-starter-security and thymeleaf-extras-springsecurity6 dependencies.
- User credentials and roles are persisted in MySQL and authenticated dynamically via CustomUserDetailsService.
- SecurityConfig declares BCryptPasswordEncoder, enables @EnableMethodSecurity and seeds initial accounts via CommandLineRunner.
- SecurityFilterChain enforces authentication on /todos/** and /api/v1/** with CSRF disabled for REST endpoints.
- Method security (@PreAuthorize) on TodoRestController provides defense-in-depth authorization.
- Delete operations (/todos/*/delete and DELETE /api/v1/todos/**) strictly require the ADMIN role.
- todos/list.html displays the authenticated user name and dynamically hides Delete for non-admins.
- Regular user gets 403 Forbidden when attempting to delete via REST; admin succeeds.